

A PROJECT REPORT

on

ClassEase

“An Academic Performance, Ranking and Feedback System for Schools”

Submitted by

Mr. Shaikh Farhan

in partial fulfilment for the award of the degree

BACHELOR OF SCIENCE (B.Sc.)

in

COMPUTER SCIENCE

under the guidance of

[Name of Project Guide]

and

[Name of Co-Guide / Mentor]

Department of Computer Science

[Name of College]

[City, State — PIN Code]

Semester [V] • Academic Year [20XX – 20XX]

CERTIFICATE

This is to certify that Mr. Shaikh Farhan of the B.Sc. (Computer Science) class, Roll No. [Roll Number] , University Seat No. [Seat Number], has satisfactorily completed the project titled “ClassEase: An Academic Performance, Ranking and Feedback System for Schools”, submitted in the partial fulfilment for the award of the Bachelor of Science degree in Computer Science during the academic year [20XX – 20XX].

Date of Submission: _____

Place: _____

Coordinator
Department of Computer Science

Project Guide
Department of Computer Science

Checked by: _____

Signature of Examiner:

College Seal

[Affix College Seal here]

[NOTE: The genuine institutional certificate must be printed on the college / department letterhead and must be signed by the coordinator, project guide and examiner before submission. The details above are placeholders only.]

DECLARATION

I, Mr. Shaikh Farhan, hereby declare that the project entitled “ClassEase: An Academic Performance, Ranking and Feedback System for Schools”, submitted in the partial fulfilment for the award of the Bachelor of Science degree in Computer Science during the academic year [20XX – 20XX], is my original work and that the project, in whole or in part, has not been submitted for the award of any other degree, diploma, associateship, fellowship or any other similar title at this or any other institution.

Signature of the Student : _____

Place : _____

Date : _____

Head / Coordinator

Department of Computer Science

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who contributed to the successful design and development of this project.

I extend my deepest gratitude to my project guide, [Name of Project Guide], and to my co-guide / mentor, [Name of Co-Guide / Mentor], for their invaluable guidance, constant encouragement and expert feedback at every stage of this work. Their insights significantly improved the quality of this project.

I would also like to thank the Head and Coordinator of the Department of Computer Science, [Name of HOD / Coordinator], and the entire faculty and staff of the department for providing the necessary resources, tools and a supportive environment that made this work possible.

My heartfelt thanks go to my family and friends for their unwavering support, patience and constructive feedback throughout the project journey. Their encouragement has been a constant source of motivation.

Finally, I wish to thank all the participants and classmates who tested the system and shared their feedback, which played an important role in refining the functionality and usability of the application.

Thank You,

Shaikh Farhan

ABSTRACT

ClassEase is a web-based academic management system designed to consolidate student performance, attendance, homework, ranking and feedback information within a unified role-based platform. The system serves three principal user categories: the institutional administration, teachers and students.

The primary motivation for the project is the comparative-performance vision on which the entire application is built. Each student is able to compare himself or herself with classmates of the same class (who scored more, in which subject, and by how much), to learn what higher-performing students do in order to improve subject-wise performance, and to compare against his or her own previous semester in order to track progress over time. These views are delivered through dedicated comparative analytics endpoints and a student-facing Performance page that presents ranking, gap analysis, semester-wise progress and head-to-head comparisons.

The system supports the complete academic workflow: authentication and role-based access control, class, teacher, student and subject management, bulk attendance marking, homework assignment, score recording with semester tracking, a composite leaderboard, announcements, a weekly timetable, a student-concern feedback workflow, and role-specific analytical dashboards with graphical representation. The feedback engine is deliberately rule-based and shallow: it derives actionable recommendations from attendance thresholds, homework completion thresholds and subject-wise performance gaps rather than from machine learning models.

Technically, ClassEase is implemented as a Laravel 13 backend API (PHP 8.3) secured with Laravel Sanctum and a role middleware, paired with a React 18 / TypeScript single-page application using Vite, Tailwind CSS and Recharts. MySQL 8 is the primary database with Redis 7 used for caching, sessions and queues. The environment is containerised with Docker and nginx. The verified development baseline records zero PHPStan (level 7) errors and twenty passing tests with eighty-one assertions.

LIST OF FIGURES

Figure Number	Figure Title
Figure 4.1	ClassEase High-Level System Architecture
Figure 4.2	Flow of Project (Process Model)
Figure 4.3	Request / Data Flow Sequence
Figure 4.4	Entity Relationship Diagram
Figure 4.5	Role Hierarchy and Authorization Design
Figure 4.6	Docker / Deployment Architecture
Figure 4.7	Comparative Performance Logic
Figure 4.8	Feedback Engine Decision Flow
Figure 4.9	Module and Navigation Structure
Figure 5.1	Login Interface (screenshot placeholder)
Figure 5.2	Management Dashboard (screenshot placeholder)
Figure 5.3	Student Performance Page (screenshot placeholder)

LIST OF TABLES

Table Number	Table Title
Table 4.1	Hardware Requirements
Table 4.2	Software Requirements
Table 4.3	Functional Requirements
Table 4.4	Non-Functional Requirements
Table 4.5	User Roles and Route-Level Permissions
Table 4.6	Database Entities
Table 4.7	Composite Score Policy Weights
Table 4.8	Feedback Engine Policy Thresholds
Table 4.9	Technologies Used
Table 5.1	Development Environment
Table 5.2	Verification and Test Results
Table 5.3	Demonstration (Development) Credentials
Appendix A.1	Detailed Role–Permission Matrix
Appendix B.1	Full-Stage Verification Summary

INDEX

CHAPTER 1: PROJECT OVERVIEW	1
1.1 Introduction.....	1
1.2 Project Background.....	1
1.3 Project Objectives	1
1.4 Scope.....	2
1.4.1 Functional Scope.....	2
1.4.2 Technical Scope	2
1.5 Target Users	2
1.6 Major Features	3
1.7 Hardware Requirements.....	3
1.8 Software Requirements	3
1.9 Assumptions.....	4
1.10 Dependencies	4
1.11 Deliverables	4
1.12 Report Organization.....	4
CHAPTER 2: INTRODUCTION AND MOTIVATION.....	6
2.1 Background	6
2.2 Existing Situation.....	6
2.3 Problem Definition.....	6
2.4 Motivation.....	7
2.5 Need for the Proposed System.....	7
2.6 Goals	7
2.7 Expected Benefits	7
2.8 Functional Perspective	8
2.9 Relevant Theoretical Concepts	8
CHAPTER 3: LITERATURE REVIEW	9
3.1 Purpose of This Review	9
3.2 Academic Performance Management Systems.....	9
3.3 Student Performance Analytics and Ranking	9
3.4 Attendance Monitoring	9
3.5 Homework and Assessment Tracking	9
3.6 Feedback Systems and Rule-Based Recommendations.....	10
3.7 Role-Based Academic Platforms and Dashboards	10

3.8 Identified Gap and Motivation.....	10
CHAPTER 4: ANALYSIS AND DESIGN	11
4.1 System Analysis.....	11
4.2 Functional Requirements	11
4.3 Non-Functional Requirements	12
4.4 User Roles	12
4.5 Use Cases	13
4.6 System Architecture.....	13
4.7 Flow of Project.....	13
4.8 Data Flow.....	15
4.9 Feasibility Study	15
4.10 Database Design.....	15
4.11 Role-Based Access Control and Authorization Design	17
4.12 Deployment Architecture.....	19
4.13 Comparative Performance Logic	19
4.13.1 Composite Leaderboard Policy.....	20
4.14 Feedback Engine Design.....	20
4.15 Module and Navigation Structure.....	22
4.16 UI / Dashboard Design.....	22
4.17 Technologies Used.....	22
CHAPTER 5: IMPLEMENTATION	23
5.1 Development Environment	23
5.2 Backend Implementation	23
5.3 Authentication and Role-Based Access Control.....	23
5.4 API Implementation.....	23
5.5 Database Implementation.....	24
5.6 Dashboard Implementation.....	24
5.7 Attendance, Homework and Scores Implementation.....	24
5.8 Leaderboard and Comparative Analytics Implementation	24
5.9 Feedback Engine Implementation.....	25
5.10 Announcements, Timetable and Concerns Implementation	25
5.11 Caching and Performance	25
5.12 Validation and Error Handling.....	25
5.13 Testing.....	26

5.14 Interface Screens	26
5.15 Demonstration Credentials.....	28
CHAPTER 6: CONCLUSION AND FUTURE SCOPE.....	29
6.1 Project Summary.....	29
6.2 Objectives Achieved	29
6.3 Major Contributions.....	29
6.4 Limitations	30
6.5 Future Scope	30
6.6 Final Conclusion	30
CHAPTER 7: REFERENCES	31
GLOSSARY	31
APPENDIX A: ROLE–PERMISSION MATRIX.....	33
APPENDIX B: PROJECT VERIFICATION / TESTING SUMMARY	35
APPENDIX C: REQUIRED SUBMISSION ATTACHMENTS.....	36

[Table of Contents is auto-generated. It is refreshed automatically during final processing before the PDF is exported.]

APPROVED PROJECT PROPOSAL

[Insert the institution-approved ClassEase project proposal here — approved and signed by the department before final submission.]

PLAGIARISM / SIMILARITY REPORT

[Insert the genuine plagiarism / similarity report issued for this project here (e.g. report generated by the institution's approved similarity-check tool).]

CHAPTER 1: PROJECT OVERVIEW

ClassEase: An Academic Performance, Ranking and Feedback System for Schools

1.1 Introduction

ClassEase is a web-based academic management system designed to consolidate student performance, attendance, homework, ranking and feedback information within a unified platform. The system is organised around three principal user roles: the institutional administration (management), teachers and students to enable efficient handling of classes, subjects, student records, attendance, homework, scores, leaderboards, announcements, timetables and student concerns.

The defining purpose of the project is its comparative-performance vision: every student is able to compare results against classmates, understand how higher-performing students differ subject-wise, and compare present performance with past semesters. This comparative capability is provided through a dedicated performance module rather than a simple score list.

1.2 Project Background

Traditional school administration relies on manual paper registers, scattered records and informal communication between administration, teachers and students. Attendance is recorded by hand, homework is assigned and tracked in isolation, and student results are rarely consolidated or analysed. Students in particular have no structured means of benchmarking themselves against classmates or their own previous performance. ClassEase was conceived to address these gaps with a single digital platform that records the academic workflow and transforms the accumulated records into meaningful, comparative insight.

1.3 Project Objectives

- To build a centralized platform for managing school operations including classes, teachers, students and subjects.
- To implement role-based access control for administration, teacher and student categories.
- To digitize attendance tracking with bulk marking and per-student daily records.
- To enable homework assignment, tracking and deadline visibility.
- To record subject-wise, semester-wise scores and compute class leaderboards on demand.
- To provide comparative performance analytics: compare with classmates, identify subject gaps to the top three, track semester progress, and support head-to-head comparison.
- To deliver role-specific analytical dashboards with graphical representation of the above data.

- To include a rule-based feedback mechanism that translates attendance, homework and performance-gap data into actionable recommendations.

1.4 Scope

1.4.1 Functional Scope

- Authentication: registration (student-role only, enforced server-side), login, logout and token-based sessions.
- Role-based access control at the level of every protected API route.
- Maintenance of teachers, classes, students and subjects including linked login accounts.
- Attendance marking and viewing by class, date and student.
- Homework creation, editing, deletion and class-wise listing.
- Score recording per student, subject, exam type and semester.
- Class leaderboard computed from score averages with exam and semester filters.
- Comparative analytics endpoints for subject comparison, gap analysis, progress tracking and head-to-head comparison.
- Announcements (general or class-specific), weekly timetable maintenance and a student-concern feedback workflow.
- Role-specific dashboards incorporating charts and notices/tasks feeds.

1.4.2 Technical Scope

- Backend: Laravel 13 (PHP 8.3) exposing a REST-style JSON API protected by Laravel Sanctum and a role middleware.
- Frontend: standalone React 18 / TypeScript single-page application built with Vite and styled with Tailwind CSS, using Recharts for visualisation.
- Data: MySQL 8 as the primary relational store; Redis 7 for caching, sessions and queues.
- Deployment: containerised with Docker and nginx; local development uses SQLite.
- Quality gates: Pint, PHPStan (level 7) and a Pest/PHPUnit test suite.

1.5 Target Users

- Administration (Management) — school administrators managing master data, statistics and global operations.
- Teachers — staff who mark attendance, assign homework, record scores and interact with their classes.
- Students — learners who view their records, timetables, announcements and comparative performance.

1.6 Major Features

- Comparative performance engine (classmate comparison, gap analysis, semester progress, head-to-head).
- Composite class leaderboard with exam and semester filters.
- Rule-based feedback engine with configurable policy thresholds.
- Role-scoped dashboards with charts (donut, bar, grouped bar, sparkline).
- Bulk attendance marking with uniqueness enforcement per student per day.
- Semester-aware score management with duplicate detection.
- Weekly timetable grid editor for a class, with per-teacher and per-student views.
- General and class-scoped announcements with a notices/tasks dashboard feed.
- Student concern lifecycle (raise, respond, resolve) with automatic resolver tracking.
- Linked login accounts for administration-created teachers and students.

1.7 Hardware Requirements

Table 4.1: Hardware Requirements

Component	Minimum Specification	Purpose
Processor	Dual-core x86-64 (64-bit), 2.0 GHz or better	Development and hosting
Memory (RAM)	8 GB (16 GB recommended)	PHP-FPM, MySQL, Redis, Node build
Storage	10 GB free disk space	Code, dependencies, database volumes
Network	LAN / internet connectivity	Client-server communication, npm/composer installs
Display	1366 × 768 or higher	Frontend usage and dashboards

1.8 Software Requirements

Table 4.2: Software Requirements

Component	Version / Detail	Role
PHP	8.3 or newer	Backend runtime
Laravel	13.x	Web framework
Laravel Sanctum	4.x	Token authentication
MySQL	8.0	Primary relational database
Redis	7.x	Cache, sessions, queues

Node.js / npm	20+ / latest	Frontend tooling
React / TypeScript	18.3.x / ≥ 5	Frontend SPA
Vite	latest	Dev server and build
Tailwind CSS	3.4.x	Styling
Recharts	$\wedge 3.10.1$	Charts
Docker / docker compose	latest	Containerised stack
Nginx	latest	Reverse proxy

1.9 Assumptions

- The institution operates single-school records (a single tenant model).
- Administration personnel have the authority to create and revoke teacher and student accounts.
- Scores, attendance and homework data are entered digitally by authorised staff.
- Internet browsers support the modern JavaScript features used by the React frontend.

1.10 Dependencies

- Composer package installation requires PHP 8.3-compatible tooling.
- Windows development requires npm.cmd invocations because PowerShell blocks npm.ps1 scripts.
- PHP static analysis and tests run inside a Docker verification container because the local XAMPP PHP (8.2.12) is below the Composer platform requirement.
- The frontend client proxies /api requests to the Laravel backend during development.

1.11 Deliverables

- Backend REST API source (Laravel 13) with migrations, models, seeders and route definitions.
- Frontend single-page application source (React 18 / TypeScript).
- Dockerized deployment definition (docker-compose and service images).
- This blackbook report together with the project journal and diagram sources.

1.12 Report Organization

Chapter 1 presents the project overview. Chapter 2 discusses the introduction and motivation for the system. Chapter 3 provides a literature review of related academic-management software and analytics approaches. Chapter 4 presents the analysis and design covering functional and non-functional requirements, roles, use cases,

architecture, database design and the comparative-performance and feedback logic. Chapter 5 explains the implementation. Chapter 6 presents the conclusion and future scope. Chapter 7 lists the references. A glossary and appendices complete the report.

CHAPTER 2: INTRODUCTION AND MOTIVATION

2.1 Background

Modern educational institutions generate large volumes of operational data — class rosters, attendance records, homework submissions and assessment marks. When these records exist only on paper or in independent spreadsheets, they cannot be related easily. The relationship between consistent attendance, prompt homework completion and examination performance is well understood in education literature; yet in most school settings this relationship is not made visible to the students or to the staff who guide them.

Assessment has two complementary purposes: certification (measuring achievement) and feedback (helping the learner improve). Systems that address only the first purpose produce marks without guidance. A system that also surfaces comparative information — how a student performs relative to classmates, where the greatest gap to higher achievers lies, and how performance changes across time — turns recorded marks into a tool for improvement.

2.2 Existing Situation

- Attendance is recorded manually, making daily registers error-prone and time-consuming.
- Homework and deadlines are not tracked centrally across classes.
- Student, teacher and class data reside in separate, unconnected records.
- Management has no immediate, consolidated view of institutional metrics.
- There is no central platform for announcements, timetables or student concerns.
- Students receive marks but no structured comparison with classmates or with their own past performance.

2.3 Problem Definition

The core problem is that valuable academic data exists but is not organised or analysed. Without consolidation and comparative analysis, neither students nor staff can act on it. Students cannot answer elementary improvement questions — “who scored more than me and in which subject?”, “what separates me from the top of my class?”, or “am I improving compared with my previous semester?”. This limits the formative value of assessment.

Formally, the problem addressed by this project is the absence of an integrated, role-secured academic platform that (i) records class, teacher, student, subject, attendance, homework and score data, (ii) derives comparative performance information from that data, and (iii) communicates actionable feedback to students.

2.4 Motivation

The motivation for ClassEase is threefold. First, the digital consolidation of academic records removes manual duplication and errors. Second, comparative analytics convert static records into usable insight, making the formative purpose of assessment explicit. Third, a role-based platform with a student-concern workflow closes the communication gap between students and administration. These motivations follow the comparative-performance vision that is the foundation of the project: compare with classmates, learn how to outscore others, and beat one's past performance.

2.5 Need for the Proposed System

- A single authoritative repository for classes, teachers, students and subjects.
- Digital, verifiable attendance records with per-student-per-day uniqueness.
- Centralized homework tracking with deadlines visible to students and teachers.
- Semester-aware score records that make time-based comparison possible.
- Automatically computed class leaderboards and rank information.
- Comparative performance views and rule-based feedback for students.
- A structured channel for announcements, timetables and student concerns.

2.6 Goals

- Deliver all planned academic modules with verified quality gates (Pint, PHPStan level 7, automated tests).
- Enforce role-based access on every authenticated route.
- Provide accurate, semester-aware comparative analytics computed on the fly from stored scores.
- Present dashboards that are informative to each role.
- Keep the feedback mechanism transparent and rule-based rather than opaque.

2.7 Expected Benefits

- Reduced administrative effort and fewer manual errors in attendance and records.
- Improved student insight through comparative benchmarks (classmates, top three, past self).
- Early identification of subjects in which a student lags the strongest peers.
- Better visibility of deadlines and institutional notices.
- Accountable concern resolution with automatic resolver tracking.
- Real-time institutional statistics for management.

2.8 Functional Perspective

Functionally, ClassEase operates as a set of vertical modules over a shared relational store. Master-data modules (teachers, classes, students, subjects) feed operational modules (attendance, homework, scores). The leaderboard and comparative analytics modules are read-only derivations over scores. The feedback engine is a thin rule layer over the derived metrics. All modules are exposed through a JSON API and consumed by a single-page client, which renders role-appropriate screens.

2.9 Relevant Theoretical Concepts

- Role-Based Access Control (RBAC): resources are protected by a role middleware so that route access is decided by the authenticated role.
- REST-style APIs: resources are exposed as stateless endpoints carrying JSON; authentication uses bearer tokens.
- ORM and the repository view: Eloquent maps database tables to domain models, allowing the analytic modules to compose queries.
- Comparative and self-referential assessment: the design follows the well-established educational principle that feedback that includes a point of comparison (class, top performers, previous self) supports improvement (Bloom, 1984) [1].
- Cache-aside pattern: Redis caches derived values while MySQL remains the source of truth.
- Composite scoring: multiple weighted indicators (academics, attendance, homework) are combined by an explicitly stated institutional policy.

CHAPTER 3: LITERATURE REVIEW

3.1 Purpose of This Review

This chapter reviews the categories of existing work that are relevant to ClassEase. The review is thematic and is written from the documented knowledge of comparable software and educational practice. Only verifiable sources are cited; a consolidated reference list appears in Chapter 7. Where specific literature has not yet been consolidated within the institution, the relevant pages are identified as requiring completion before final submission.

3.2 Academic Performance Management Systems

Academic management software is a mature category. Products in this space commonly provide student records, class schedules, attendance and grade books, and increasingly expose parent portals. The limitation of many commercial systems is that they are oriented toward administration and reporting rather than toward the individual student's comparative learning view. ClassEase differentiates itself by making share-of-class, top-three gaps and semester-over-semester change first-class features of the student experience rather than management-only reports.

3.3 Student Performance Analytics and Ranking

Ranking students within a cohort is used widely as a motivational instrument. The education literature cautions that ranking can be demotivating if it emphasises position without guidance on improvement; the constructive approach pairs ranking with the information needed to act (cf. the broader evidence that success-criterion feedback supports learning [2]). ClassEase addresses this by pairing the class rank with gap analysis and subject-level diagnosis, so that a rank value is accompanied by an explanation of where improvement is most achievable.

3.4 Attendance Monitoring

Attendance is a well-documented correlate of academic achievement. Electronic attendance systems replace manual registers with structured records, typically enforcing one record per student per day and allowing summary statistics. ClassEase implements this enforcement as a database-level uniqueness constraint and feeds attendance rates into its feedback rules. This mirrors common practice in lightweight classroom management tools while keeping the data integration in one database.

3.5 Homework and Assessment Tracking

Centralized homework tracking with deadlines addresses the classic failure of paper-based assignment management: deadlines are lost and completion is invisible. The literature on formative assessment emphasizes frequent, low-stakes tasks with prompt feedback (the “90% plus time-on-task” relation to mastery learning is well established [1]). ClassEase therefore records homework at class and subject level, exposes deadlines

with an overdue indicator, and treats homework completion as one component of its composite view of a student.

3.6 Feedback Systems and Rule-Based Recommendations

Automated feedback ranges from rule-based decision systems to data-driven machine-learning systems. Rule-based engines are transparent, explainable and cheap to operate: a small set of thresholds converts measured indicators into recommendations. Machine-learning systems may be more adaptive but are opaque and require substantial labelled data. For an institutional academic setting, transparency is a primary requirement, so ClassEase deliberately adopts a shallow, rule-based feedback engine built on attendance, homework completion and subject-gap thresholds; it is not claimed to be an artificial-intelligence system.

3.7 Role-Based Academic Platforms and Dashboards

Multi-role platforms enforce that each user sees only data permitted by his or her role. The standard implementation is route-level middleware combined with object-level checks. Dashboards that summarise metrics per role (administrative statistics, teacher class-performance, student self-comparison) are a common and effective presentation device. ClassEase follows this pattern with role-scoped dashboard feeds and chart-based presentation.

3.8 Identified Gap and Motivation

Existing approaches either store data without comparative analysis, or provide analytics without data collection, or restrict comparative views to administration. The identified gap is an integrated platform that (a) records the full academic workflow and (b) exposes comparative, semester-aware analytics and rule-based feedback directly to students. ClassEase is designed to fill that gap. The technical realisation follows the documented Laravel and React ecosystem [3][4][5][6][7][8][9][10][11][12][13][14].

CHAPTER 4: ANALYSIS AND DESIGN

4.1 System Analysis

The system was analysed from the perspective of the three user roles and the data they exchange. A typical cycle begins with an administrator creating the master data (teachers, classes, students, subjects). Teachers then operate the academic modules (attendance, homework, scores). Students consume the resulting records through dashboards, timetables and the comparative performance views. Two derived components complete the analysis: the composite leaderboard and the rule-based feedback engine.

4.2 Functional Requirements

Table 4.3: Functional Requirements

Requirement	Description
Authentication	Registration (student role enforced server-side), login, logout, current-user retrieval
Role-based access	Every protected route gated to role:admin, role:admin,teacher or role:admin,teacher,student
Master data CRUD	Teachers, classes, students and subjects with admin-only writes
Linked accounts	Teacher/student accounts created by admin receive a matching login (email + password)
Attendance	Bulk marking per class and date (present/absent/late), one record per student per day, view by class/date/student
Homework	Create (teachers scoped to their own subjects), list by class, edit, delete (teacher own only), overdue indicator
Scores	Record per student/subject/exam_type/semester; duplicate detection; update and delete
Leaderboard	Rank students of a class by overall average percentage with exam and semester filters
Comparative analytics	Subject comparison, gap-to-top-three analysis, semester progress, head-to-head comparisons
Feedback	Rule-based recommendations from attendance, homework completion and subject gaps
Announcements	Post general or per-class notices; list, edit, delete; role-scoped feeds
Timetable	Per-class weekly grid (periods $\times$ days), bulk save, teacher and student views
Concerns	Student raise, admin/teacher manage status and reply, automatic resolver tracking
Dashboards	Role-scoped statistics, feeds and charts

4.3 Non-Functional Requirements

Table 4.4: Non-Functional Requirements

Category	Requirement
Security	Password hashing (bcrypt), bearer-token authentication, route-level RBAC, validated and least-privilege data exposure
Performance	Leaderboard and comparative analytics computed on the fly over indexed score records; Redis caching for derived data
Reliability	Transactions for operations that create linked records; idempotent seeders; unique-constraint enforcement in the database
Maintainability	Pint code style, PHPStan level 7, typed controllers, structured module layout
Testability	Pest test suite covering authentication, RBAC, workflows and analytics
Usability	Role-specialised navigation, clear dashboards, consistent form patterns
Portability	Dockerised deployment; local SQLite fallback for development

4.4 User Roles

Three roles are defined: admin (management), teacher and student. Route groups enforce access at the transport layer, and controller-level guards (for example `isStudentForbidden`) enforce object-level restrictions, such as a student reading only his or her own records or only the roster of his or her own class.

Table 4.5: User Roles and Route-Level Permissions

Feature Area	admin	teacher	student
Dashboard statistics	●	—	—
Teacher/Class/Student/Subject CRUD	●	—	—
Class roster, teacher & subject reads	●	●	own class only
Attendance / Homework / Scores / Announcements (write)	●	●	—
Raise a concern	—	—	●
Read endpoints (own data, lists, leaderboard)	●	●	●
/me and /logout	●	●	●

4.5 Use Cases

- Admin use case: authenticate and manage teachers, classes, students, subjects; view statistics; manage announcements; resolve concerns.
- Teacher use case: authenticate; view own profile and subjects; mark attendance; record scores; assign homework; manage timetable and announcements; respond to concerns.
- Student use case: authenticate; view own records, class schedule, announcements and homework; raise concerns; explore comparative performance (class, rank, focus, progress, head-to-head).
- Cross-cutting use case: any authenticated role retrieves his or her profile and logs out.

4.6 System Architecture

The system follows a client-server architecture. A standalone React single-page application consumes a Laravel JSON API. All protected endpoints require a Sanctum bearer token and pass through the role middleware. The architecture is shown in Figure 4.1.

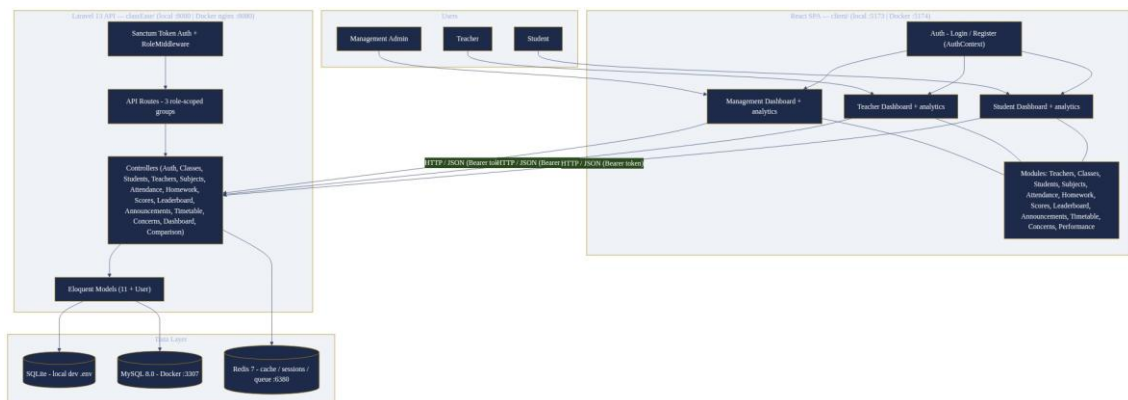


Figure 4.1: ClassEase High-Level System Architecture

4.7 Flow of Project

The operational flow of the system proceeds from master-data creation through the academic modules to the analytical and feedback views, as illustrated in Figure 4.2.

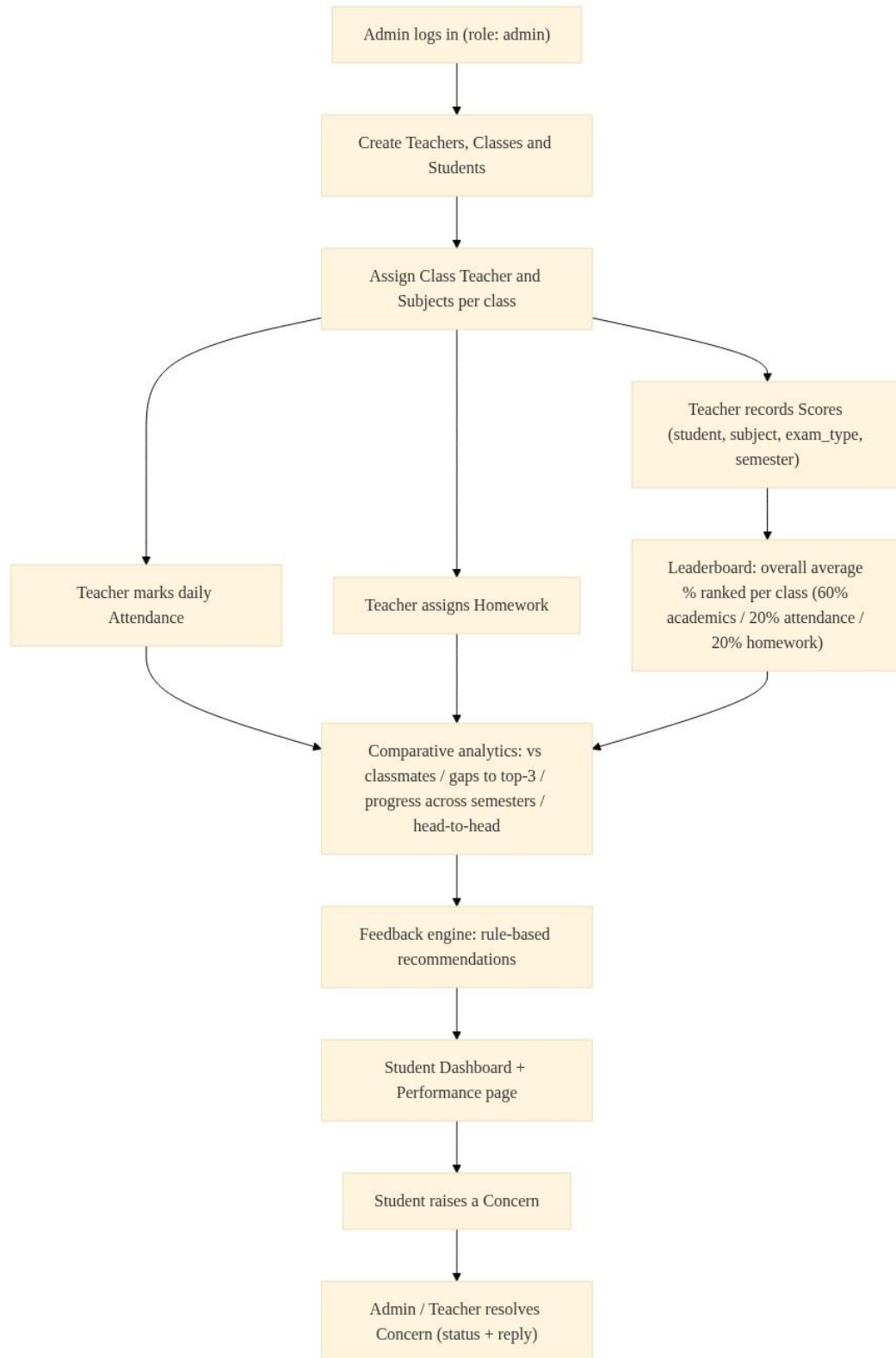


Figure 4.2: Flow of Project (Process Model)

4.8 Data Flow

A request flows from the browser to the Vite development server, which proxies `/api/` requests to the Laravel backend (through nginx in the Docker deployment). The Laravel router authenticates the token, checks the role, dispatches to a controller, and the controller returns JSON data that the client renders. Figure 4.3 shows this sequence.

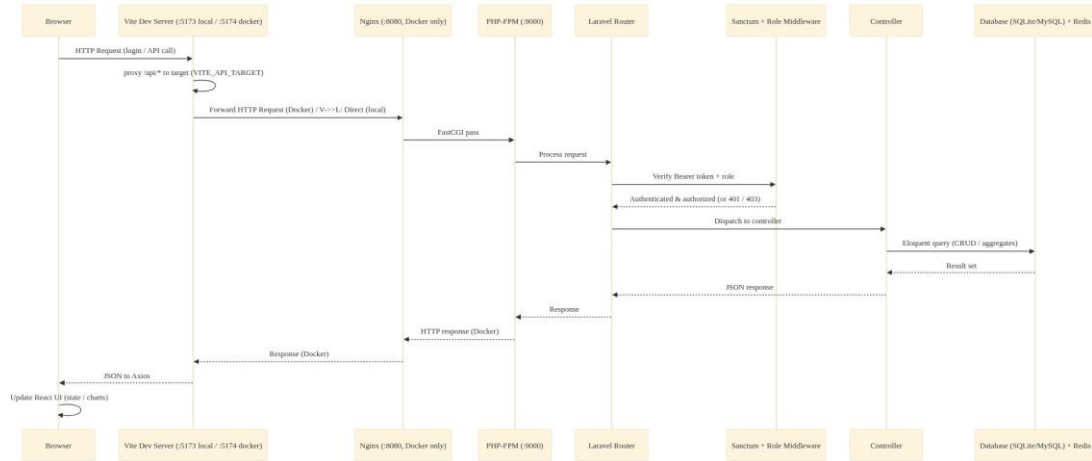


Figure 4.3: Request / Data Flow Sequence

4.9 Feasibility Study

- Technical feasibility: the Laravel/React/MySQL/Redis stack is mature, freely available and well documented; a Dockerised setup removes environment variability. Feasible.
- Operational feasibility: the three-role interaction model matches the working practices of schools; role-specific pages reduce training effort. Feasible.
- Economic feasibility: all core technologies are open source; development and hosting cost are limited to infrastructure. Feasible.

4.10 Database Design

The database is relational and normalised around the master entities. The entity relationship diagram in Figure 4.4 captures the entities, attributes and relationships.

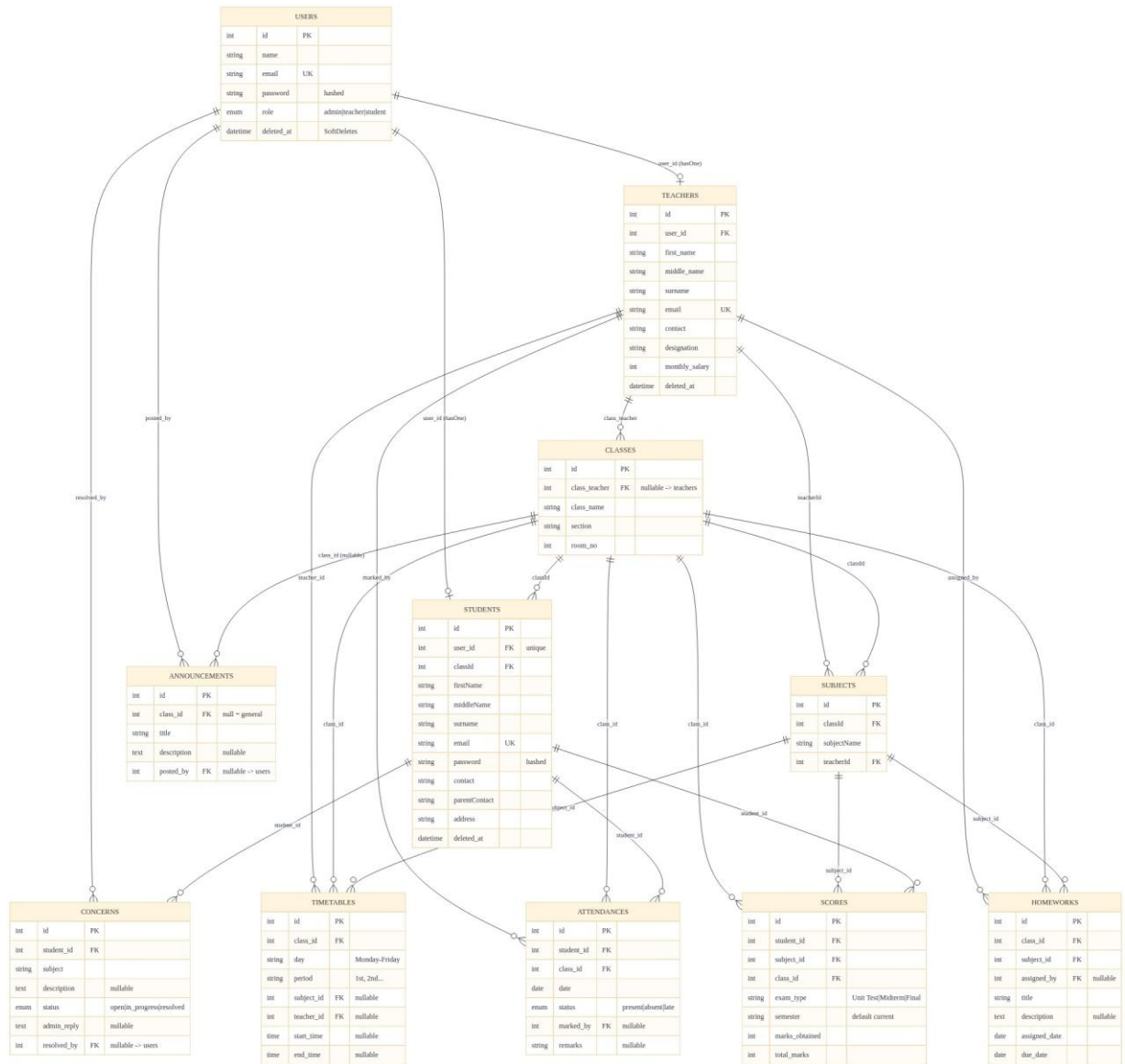


Figure 4.4: Entity Relationship Diagram

Table 4.6 lists the principal entities.

Table 4.6: Database Entities

Entity	Primary Reference	Key Attributes
users	Authentication	name, email, password, role, deleted_at
teachers	Teacher master data	first_name, middle_name, surname, email, contact, designation, monthly_salary, user_id
classes	Class master data	class_name, section, room_no, class_teacher
students	Student master data	firstName, middleName,

		surname, email, password, contact, parentContact, address, classId, user_id
subjects	Subjects of a class	subjectName, classId, teacherId
attendances	Daily presence	student_id, class_id, date, status, marked_by, remarks
homeworks	Assignments	class_id, subject_id, assigned_by, title, description, assigned_date, due_date
scores	Assessment marks	student_id, subject_id, class_id, exam_type, semester, marks_obtained, total_marks
announcements	Notices	class_id, title, description, posted_by
timetables	Weekly schedule	class_id, day, period, subject_id, teacher_id, start_time, end_time
concerns	Student feedback	student_id, subject, description, status, admin_reply, resolved_by

4.11 Role-Based Access Control and Authorization Design

Authorization is enforced at two levels. The role middleware evaluates the token holder's role against route groups: role:admin, role:admin,teacher and role:admin,teacher,student. Object-level rules are implemented in controllers, for example restricting a student to his or her own records, to the roster of his or her own class, and to comparisons involving himself or herself within the same class. Figure 4.5 shows the role hierarchy.

Management (Admin) - Full Access

Classes / Teachers / Students /
Subjects CRUD

Dashboard stats + analytics

Attendance - mark & view

Homework CRUD

Scores CRUD + Leaderboard

Announcements + Timetable

Figure 4.5: Role Hierarchy and Authorization Design

4.12 Deployment Architecture

The application is containerised with Docker Compose. Services are described in Figure 4.6. Host ports were retuned so that the stack does not conflict with local services: MySQL maps to host port 3307, Redis to 6380, nginx to 8080 and the client Vite server to 5174.

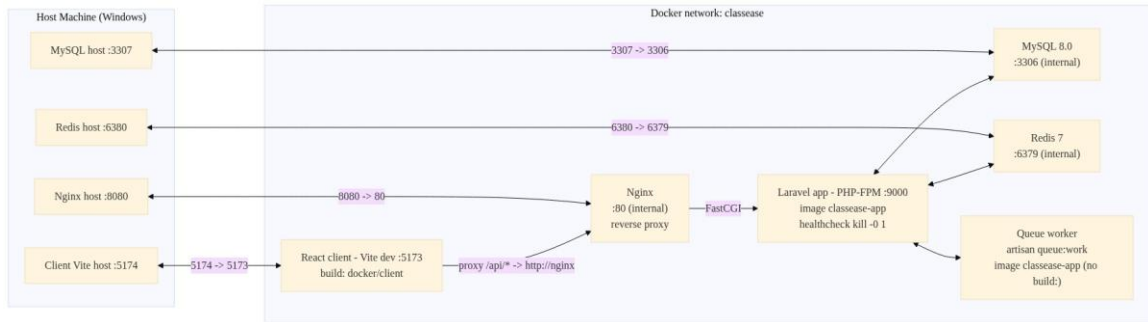


Figure 4.6: Docker / Deployment Architecture

4.13 Comparative Performance Logic

The comparative-performance module is implemented by a dedicated backend controller with four read-only endpoints over the scores table. All comparisons are computed on the fly; no derived tables are stored.

- Subject comparison: per-student marks and percentages for a chosen class, subject and semester, marking the requester's own row (is_you) and returning the class average, minimum and maximum.
- Gap analysis: for each subject of a student, the student's percentage is compared with the average of the top three performers and with the class average; gaps are sorted in descending order so the biggest opportunity appears first.
- Progress tracking: for each subject, the current semester is compared with the most recent previous semester (determined by latest record date), yielding a delta and a trend (up / down / same) together with a summary of improved and declined counts.
- Head-to-head: two students are compared subject by subject for a chosen semester; the response reports percentages, the delta, the leader (A / B / tie / n-a) and a summary of wins; rows are sorted by absolute delta.

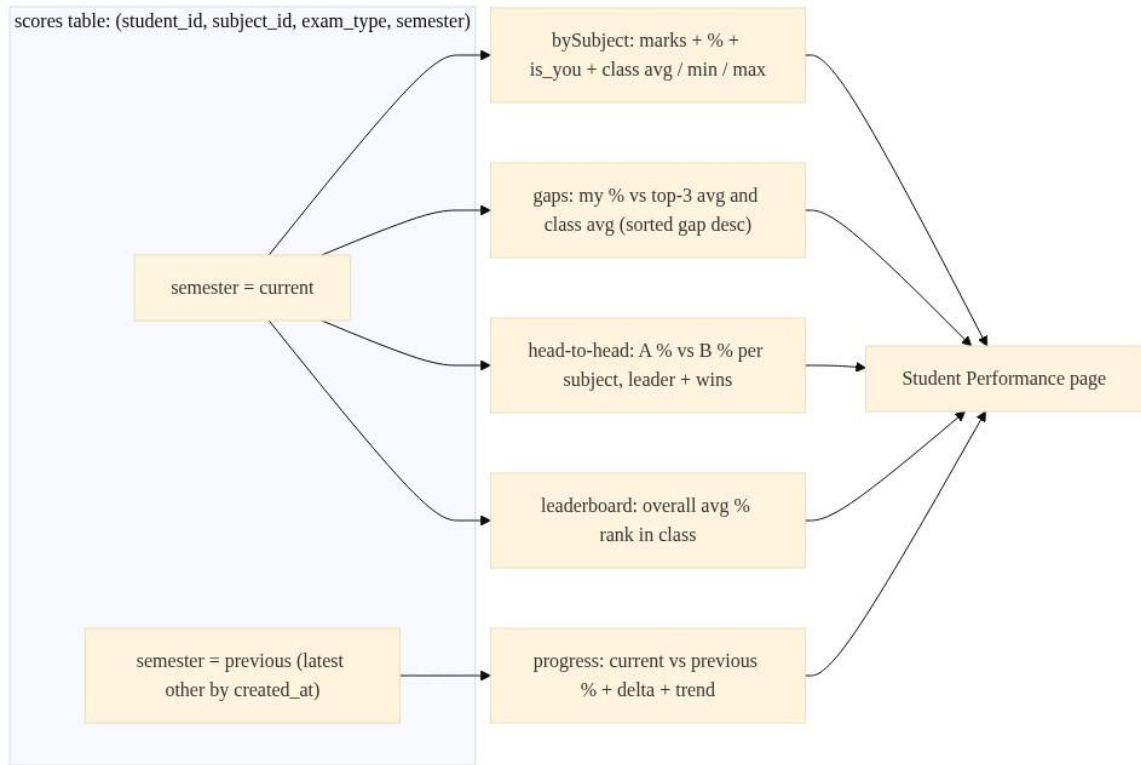


Figure 4.7: Comparative Performance Logic

4.13.1 Composite Leaderboard Policy

For ranking purposes the institution adopts a composite score that combines three components with explicitly stated weights. These weights are institutional policy constants for this project, not scientifically established optima. They are documented here so that they may be adjusted as policy.

Table 4.7: Composite Score Policy Weights

Component	Weight
Academics (average percentage across assessed subjects)	60%
Attendance rate	20%
Homework completion	20%

4.14 Feedback Engine Design

The feedback engine is deliberately rule-based and shallow. It does not employ artificial intelligence or machine learning. Given a student's recorded indicators, a small set of policy thresholds produces a recommendation that is shown in the student's performance views. The decision rules are visualised in Figure 4.8 and the thresholds are stated in Table 4.8.

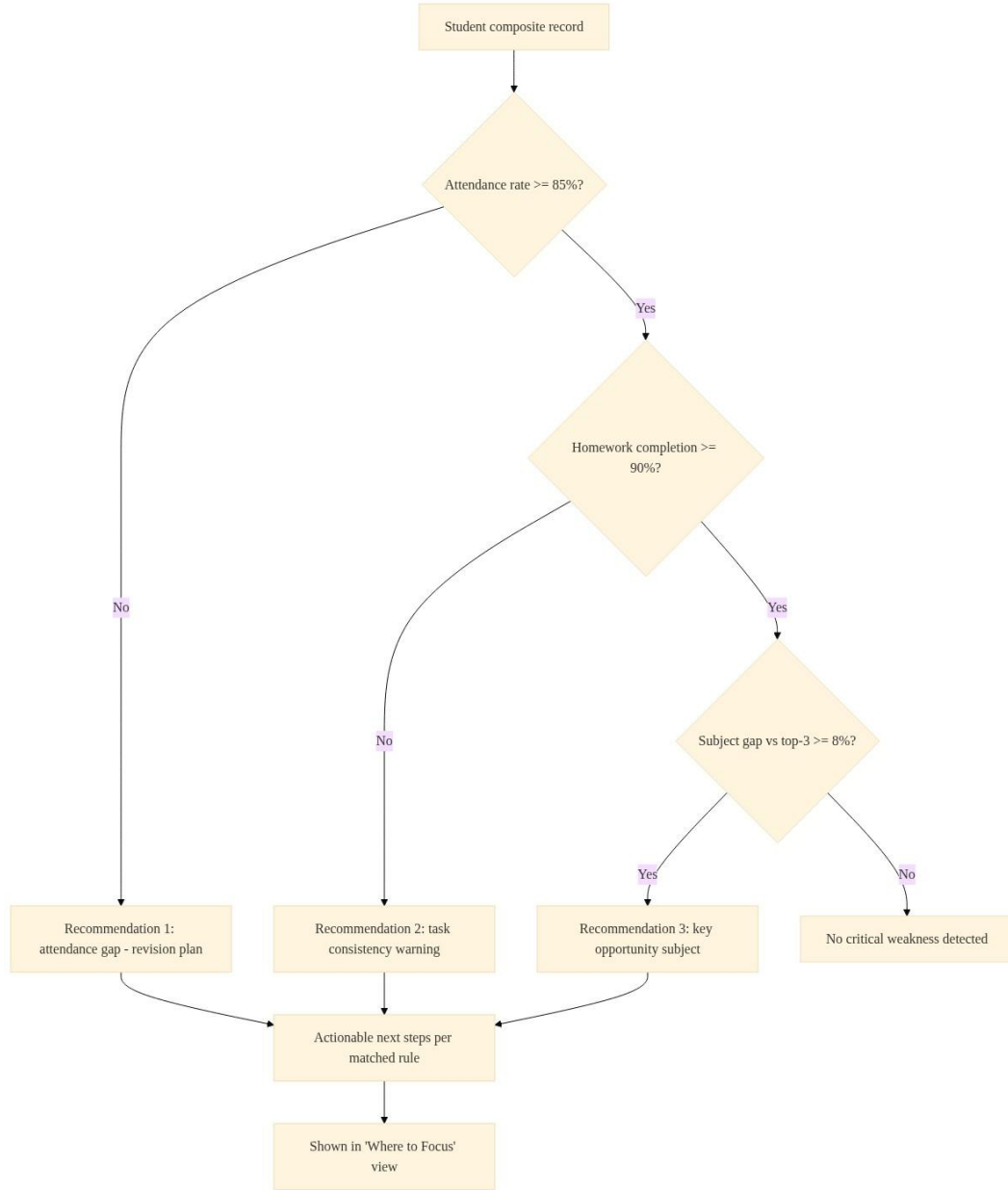


Figure 4.8: Feedback Engine Decision Flow

Table 4.8: Feedback Engine Policy Thresholds

Indicator	Threshold / Condition	Generated Recommendation
Attendance rate	Below 85%	Attendance gap warning with a revision-plan recommendation
Homework completion	Below 90%	Task-consistency warning
Subject gap to top-3 average	Equal to or greater than 8 percentage points	Identify the subject as a key opportunity area
No rule triggered	—	No critical weakness detected

4.15 Module and Navigation Structure

The single-page application organises navigation around the authenticated role. The module and navigation structure is shown in Figure 4.9.

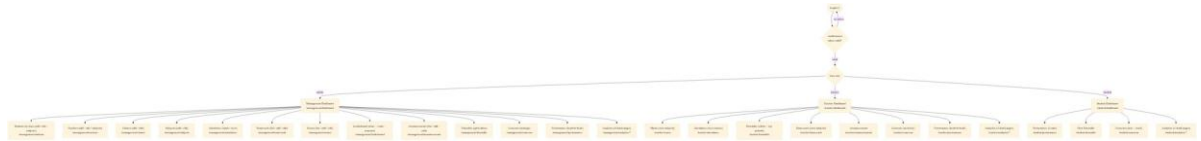


Figure 4.9: Module and Navigation Structure

4.16 UI / Dashboard Design

Each role receives a dedicated dashboard. The student dashboard presents a class-rank ring, an attendance breakdown, subject-wise percentage bars, a comparison of the student against the class average and the top three, a progress-over-time chart and a homework overview. The teacher dashboard presents attendance summaries, class performance by subject and per-student performance. The management dashboard presents school-wide counts, students-per-class, class performance and score distribution. Charts are drawn with Recharts and are organised in a responsive, card-based layout.

4.17 Technologies Used

Table 4.9: Technologies Used

Purpose	Technology
Backend framework	Laravel 13 (PHP 8.3)
Authentication	Laravel Sanctum (bearer tokens, bcrypt hashing)
Primary database	MySQL 8
Caching / sessions / queues	Redis 7 (cache-aside; MySQL remains the source of truth)
Frontend	React 18, TypeScript, Vite
Styling	Tailwind CSS 3.4
Charts	Recharts
Testing	Pest (PHPUnit based)
Static analysis	PHPStan level 7, Pint
Deployment	Docker, docker compose, nginx, PHP-FPM

CHAPTER 5: IMPLEMENTATION

5.1 Development Environment

Table 5.1: Development Environment

Item	Setting
Operating system	Windows (host); Linux containers for verification and deployment
Local PHP	XAMPP PHP 8.2.12 (Pint runs locally)
Verification PHP	Docker container with PHP 8.3 (PHPStan, artisan, tests)
Backend	Laravel 13, SQLite locally / MySQL in Docker
Frontend	Vite dev server on port 5173 (local) and 5174 (Docker)
Package managers	Composer 2 (backend), npm (frontend)
Container tooling	Docker Desktop (daemon v29.4.0)

5.2 Backend Implementation

The backend is organised as models, controllers and route definitions. Eloquent models map the database tables described in Chapter 4. Controllers expose the resource operations, and three role-scoped groups in the API route file apply the role middleware.

Two implementation details are noteworthy. First, the Homework model explicitly declares its table name because Eloquent treats “homework” as an uncountable noun:

```
protected $table = 'homeworks';
```

Second, the rank values of the leaderboard are injected into the response collection during mapping (previously a by-reference loop over an Eloquent collection never wrote the rank back, so ranks were absent from the JSON). Combined with ordered score aggregation this yields a correct, exam- and semester-aware ranking.

5.3 Authentication and Role-Based Access Control

Authentication uses Laravel Sanctum bearer tokens. Registration always creates a student account: the role is forced server-side and a client-supplied role is ignored. Login verifies credentials against the users table and issues a token. The role alias (RoleMiddleware) then gates every protected route. Admin-created teachers and students receive linked login accounts within a transaction; updates keep the linked login in sync, and deletion revokes the login (soft-deletes on users). Email uniqueness is validated against non-deleted rows only, which permits email reuse after deletion.

5.4 API Implementation

Routes are declared in routes/api.php under three role-scoped middleware groups plus the public authentication endpoints. Route ordering is significant: literal segments such as

/student/me and /concerns/student are declared before parameterised segments such as /student/{id} so that the router does not swallow them. The API surface includes authentication, master data, attendance, homework, scores, leaderboard, announcements, timetable, concerns, comparative performance and the dashboard feed.

5.5 Database Implementation

The schema is created through migrations. Notable constraints include the composite uniqueness on scores (student_id, subject_id, exam_type, semester), the per-day uniqueness on attendance (student_id, date), the week-slot uniqueness on timetables (class_id, day, period), and soft deletes on users, students and teachers. A dedicated migration makes the homework assignment owner nullable, allowing administration to create homework records without a linked teacher profile.

5.6 Dashboard Implementation

The three role dashboards were rebuilt with Recharts. The student dashboard resolves the authenticated user's own student row through a dedicated endpoint (user identifiers differ from student identifiers in the seed data) and charts the student's rank, attendance, subject percentages, comparisons and progress. The teacher dashboard resolves the teacher's own profile in the same way. The management dashboard summarises school-wide counts and class performance. Every dashboard additionally includes a notices and tasks feed scoped to the role.

5.7 Attendance, Homework and Scores Implementation

- Attendance: a bulk endpoint marks one record per student per class and date; the database enforces uniqueness; records carry a status (present / absent / late), a marker (teacher) and optional remarks.
- Homework: creation validates class, subject, title and dates; teachers may only assign within their own subjects; editing and deletion are restricted to the assigning teacher; a class-wise listing shows deadlines and overdue state.
- Scores: creation validates student, subject, class, exam type, semester, and marks; a composite-unique check returns HTTP 409 for duplicates; reads expose only safe columns and present marks and percentages.

5.8 Leaderboard and Comparative Analytics Implementation

- The leaderboard endpoint aggregates score sums (marks obtained over total marks) per class, applies optional exam and semester filters, orders descending and injects the rank into each entry.
- The subject-comparison endpoint computes per-student percentages and the class minimum, maximum and average for the selected subject and semester.
- Gap analysis computes, per subject, the gap between the student's percentage, the top-three average and the class average; results are sorted by descending gap.

- Progress tracking compares the “current” semester with the latest other semester by record date and produces per-subject deltas and trends.
- Head-to-head compares two students across the union of their subject records for a semester and returns the leader per subject plus a win summary.
- Student access is guarded so that a student can only read his or her own analytic results and own-class comparisons.

5.9 Feedback Engine Implementation

The feedback engine is implemented as a shallow, rule-based computation over the derived indicators described in Chapter 4. Its recommendations are surfaced in the student “Where to Focus” view. The engine evaluates the attendance rate against the 85% policy threshold, homework completion against the 90% threshold, and the per-subject gap to the top-three average against the 8-point threshold, generating the corresponding recommendations listed in Table 4.8. The rules are transparent and configurable as policy constants.

5.10 Announcements, Timetable and Concerns Implementation

- Announcements: class-scoped records with a null class denoting a general announcement; students see their own class plus general notices; the dashboard feed is role-scoped.
- Timetable: a bulk save operation idempotently updates the whole class grid (periods $\times$ days), deleting removed slots and auto-filling the teacher from the subject; read views exist for class and teacher.
- Concerns: a student raises a concern whose subject and description are recorded and whose student is resolved from the authenticated account; staff update the status and reply, and the resolver is recorded automatically; students are scoped to their own concerns.

5.11 Caching and Performance

Redis is used for caching, sessions and queues in the deployed stack; the application follows a cache-aside approach in which MySQL remains the source of truth. Leaderboards and comparative analytics are computed over indexed score records, and no redundant derived tables are maintained.

5.12 Validation and Error Handling

- Server-side validation is applied on every create and update operation; controller methods are typed with explicit return types to keep static analysis precise.
- Duplicate detection returns HTTP 409 for repeated composite records (scores) and validation failures return 422 with field-scoped messages.
- Role and ownership violations return 403; missing records return 404.

- Transactions wrap multi-record operations such as account creation and timetable bulk saves.
- Relations that controllers eager-load are declared on the models, keeping both runtime and static analysis correct.

5.13 Testing

Testing is performed with Pest under the development verification container. Environment overrides force an isolated, in-memory SQLite database for the test run. The suite covers authentication, role-based access, the linked-account lifecycle, dashboard feed scoping, teacher workflows and a leaderboard ranking. Table 5.2 records the verified results at the time this report was prepared.

Table 5.2: Verification and Test Results

Check	Tool / Command	Result (latest baseline)
PHP static analysis	PHPStan level 7	0 errors
Automated tests	Pest (artisan test)	20 passed / 81 assertions
PHP code style	Pint	Passed
JavaScript lint	ESLint	Passed
JavaScript format	Prettier	Passed
JavaScript types	tsc --noEmit	Passed
Frontend build	Vite + TypeScript	Passed
Full-stack smoke	docker compose + API/login/me/classes	Passed

5.14 Interface Screens

The representative screens of the application are presented below. The figures are placeholders until actual captures from the running application are inserted before submission.

Figure 5.1 shows the login interface, which is the entry point for all roles.

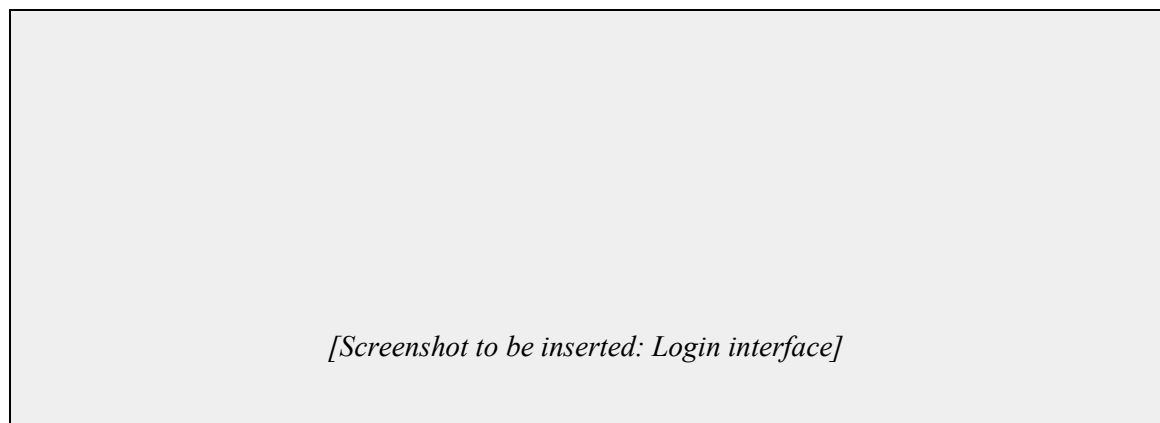




Figure 5.1: Login Interface

Figure 5.2 shows the management dashboard with statistical cards and charts.

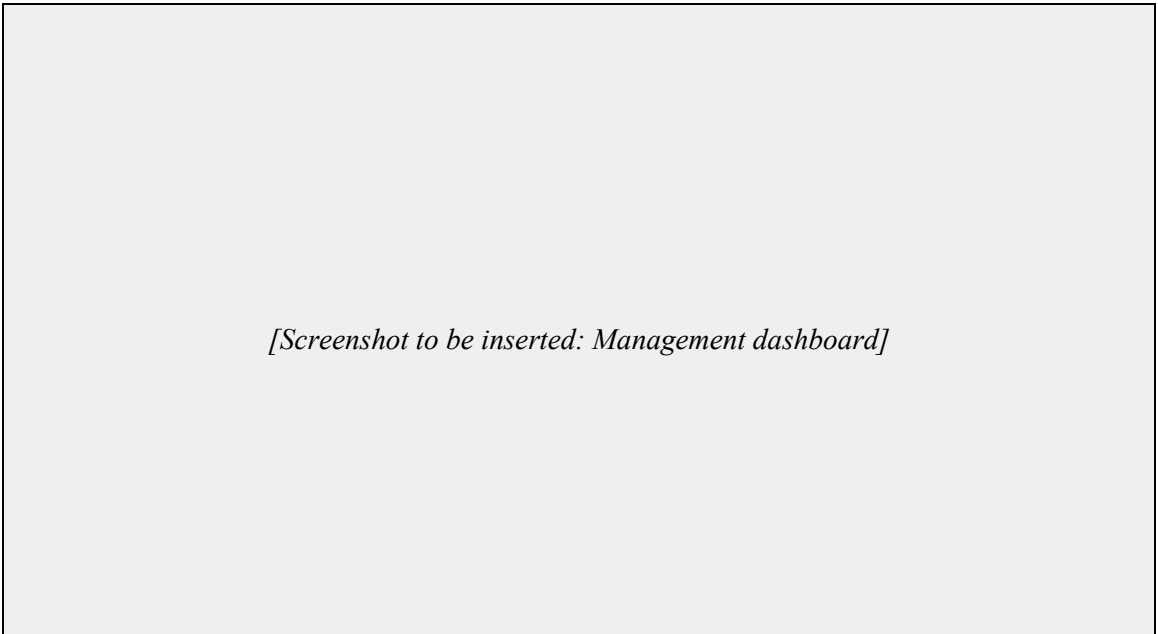


Figure 5.2: Management Dashboard

Figure 5.3 shows the student performance page with the comparative views.

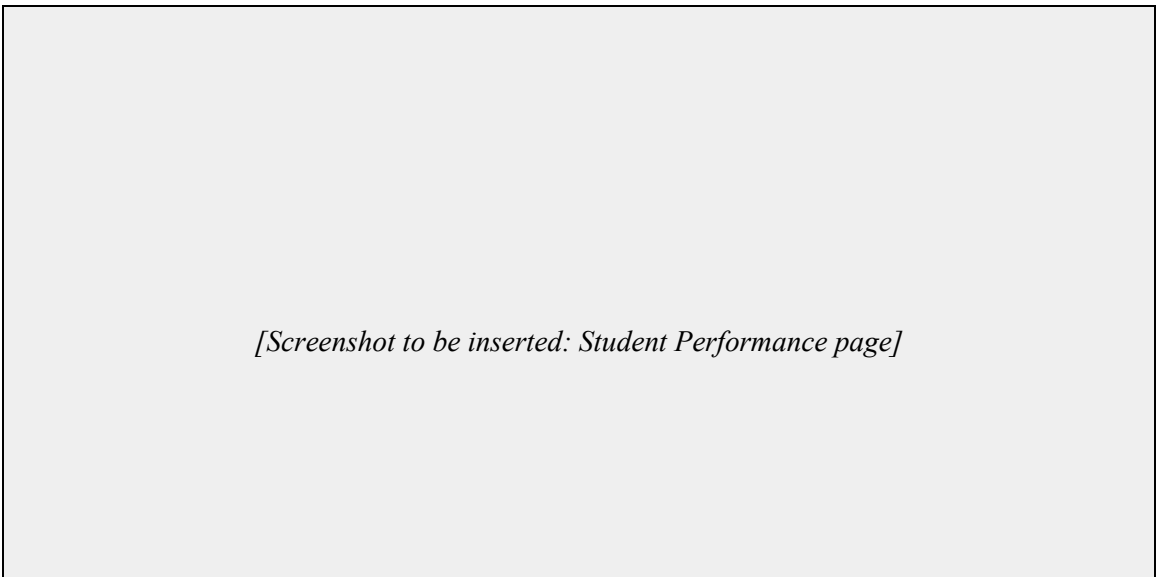




Figure 5.3: Student Performance Page

5.15 Demonstration Credentials

The following development accounts are used to demonstrate the system. These are development and demonstration credentials only and must never be used in a production environment.

Table 5.3: Demonstration (Development) Credentials

Role	Email	Password
Admin	admin@classease.com	ClassEase@123 (DEV ONLY)
Teacher	teacher1@classease.com, teacher2@classease.com, teacher3@classease.com	ClassEase@123 (DEV ONLY)
Student	student1@classease.com ... student6@classease.com	ClassEase@123 (DEV ONLY)

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Project Summary

ClassEase was designed and implemented as a role-secured academic platform that records the school workflow and converts the accumulated records into comparative, semester-aware insight for students. The system integrates master data, attendance, homework, scores, leaderboards, announcements, timetables and concerns, and derives from the scores a set of comparative analytics supported by a rule-based feedback engine.

6.2 Objectives Achieved

- A centralized platform for school operations was delivered across all planned modules.
- Role-based access control was enforced on every protected endpoint.
- Attendance tracking with bulk marking and per-day uniqueness was implemented.
- Homework assignment and tracking with deadlines and overdue indicators was completed.
- Semester-aware score management with a composite-leaderboard view was implemented.
- Comparative performance analytics (classmates, gaps, progress, head-to-head) were delivered to students.
- A transparent, rule-based feedback engine with clearly stated policy thresholds was provided.
- Role-specific analytical dashboards with charts were completed for all three roles.
- The quality gates were satisfied: zero PHPStan errors, a passing test suite, and passing lint, format, type and build checks.

6.3 Major Contributions

- A comparative-performance design that pairs every rank or result with actionable comparison (class, top three, past self).
- Semester-aware data model and analytics enabling time-based comparison.
- A layered authorization design combining route-level roles and object-level guards.
- A documented composite-score and feedback-threshold policy that institutions can tune.
- A Dockerised, reproducible toolchain with a stable verification process.

6.4 Limitations

- The feedback engine is rule-based and intentionally shallow; it does not learn from outcomes and does not use artificial-intelligence or machine-learning techniques.
- Seed and demonstration data are synthetic; conclusions about general school workloads are limited.
- There is no offline mode; the client depends on connectivity to the backend.
- Ranking can be psychologically sensitive; the system mitigates this with accompanying guidance but does not eliminate the consideration.
- Composite weights (60/20/20) and feedback thresholds are institutional policy choices and are not empirically validated optima.
- Object-level hardening is complete for student self-scoping; some teacher write routes rely primarily on the user interface for subject scoping.

6.5 Future Scope

- Code-splitting the chart library to reduce the initial bundle size.
- A Grade/Grade-Point engine built on the semester-aware score records.
- Email or SMS notifications for homework deadlines and concern replies.
- Report export (PDF/Excel) for scores, attendance and leaderboards.
- A parent portal providing read-only visibility into a student's records.
- Aphoristic model improvement of the feedback rules with more historical data.
- Deeper, data-driven feedback that extends the current rule set.

6.6 Final Conclusion

ClassEase fulfils its stated objective of consolidating academic records and exposing comparative, semester-aware analytics to students in a secure, role-based platform. The system is functionally complete across its planned modules and passes the verified quality gates. It does not claim to solve every problem of educational analytics; its feedback engine is explicitly rule-based and its scoring policy is an institutional choice. Within those honest boundaries, ClassEase provides a coherent, reproducible and extensible foundation for comparative academic performance, ranking and feedback in a school setting.

CHAPTER 7: REFERENCES

- [1] Bloom, B. S. (1984). The 2 Sigma Problem: The Search for Methods of Group Instruction as Effective as One-to-One Tutoring. *Educational Researcher*, 13(6), 4–16.
- [2] Hattie, J. (2009). *Visible Learning: A Synthesis of Over 800 Meta-Analyses Relating to Achievement*. Routledge.
- [3] Laravel Documentation. Laravel 13.x. Available: <https://laravel.com/docs>
- [4] Laravel Sanctum Documentation. Available: <https://laravel.com/docs/sanctum>
- [5] React Documentation. Available: <https://react.dev>
- [6] TypeScript Documentation. Available: <https://www.typescriptlang.org/docs>
- [7] Vite Documentation. Available: <https://vitejs.dev>
- [8] Tailwind CSS Documentation. Available: <https://tailwindcss.com/docs>
- [9] Recharts Documentation. Available: <https://recharts.org>
- [10] MySQL 8.0 Reference Manual. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [11] Redis Documentation. Available: <https://redis.io/docs>
- [12] PHPStan Documentation. Available: <https://phpstan.org>
- [13] Pest PHP Testing Documentation. Available: <https://pestphp.com/docs>
- [14] Docker Documentation. Available: <https://docs.docker.com>

Note regarding the literature review: the sources above are the verifiable references currently consolidated for this project. Any additional institutional reference list should be merged into this chapter before submission.

GLOSSARY

API: Application Programming Interface; the set of endpoints through which the client and server communicate.

RBAC: Role-Based Access Control; the enforcement of permissions according to a user's assigned role.

CRUD: Create, Read, Update, Delete; the four basic operations on persistent data.

Redis: An in-memory data structure store used here for caching, sessions and queues.

Sanctum: Laravel's lightweight token-based authentication package used for API authentication.

Composite Score: A weighted combination of academics, attendance and homework used for class ranking under institutional policy.

Cache-Aside: A caching pattern in which the application checks the cache, then the source of truth (MySQL) on a miss, and populates the cache.

REST: Representational State Transfer; a stateless, resource-oriented architectural style for web services.

ORM: Object-Relational Mapping; the technique of mapping database tables to application objects (Eloquent).

SPA: Single-Page Application; a client-side application that updates the view without full page reloads.

Leaderboard: A ranked list of students of a class ordered by their overall average percentage.

Gap Analysis: The comparison of a student's subject percentage with the top-three and class averages.

Head-to-Head: A per-subject comparison between two students for a chosen semester.

Semester: A free-form academic period label attached to score records, used for time-based comparison.

Pest: A testing framework built on PHPUnit, used for the automated test suite.

PHPStan: A static analysis tool for PHP, configured here at level 7.

APPENDIX A: ROLE–PERMISSION MATRIX

The table below maps the feature areas to the three roles at the route level, as implemented by the role middleware groups and object-level guards.

Table A.1: Detailed Role–Permission Matrix

Feature Area / Route Group	admin	teacher	student
Dashboard statistics (/dashboard/stats)	•	—	—
Classes CRUD	•	—	—
Teachers CRUD (+ linked login)	•	—	—
Students CRUD (+ linked login)	•	—	—
Subjects CRUD	•	—	—
Teacher detail / own profile (/teacher/me)	•	•	—
Teacher subjects	•	•	—
Subjects of a class	•	•	—
Attendance - mark / update / view by date	•	•	—
Homework - create / update / delete	•	• (own subjects)	—
Scores - record / update / delete	•	•	—
Announcements - post / update / delete	•	•	—
Timetable - save / update / delete	•	•	—
Concerns - manage / respond / delete	•	• (no delete)	raise only
Student /me identity	•	•	•
Student roster of a class	•	•	own class only
Attendance / homework / scores reads	•	•	• (own / own class)
Leaderboard read	•	•	•
Comparative performance reads	•	•	own + own-class H2H
Announcements / timetable reads	•	•	•
Dashboard feed (/dashboard/feed)	•	•	•
/me and /logout	•	•	•

APPENDIX B: PROJECT VERIFICATION / TESTING SUMMARY

This appendix records the verification stages applied to the project at the time of writing. The full commands and environment notes are maintained in the project checkpoint documentation.

Table B.1: Full-Stage Verification Summary

Stage	Result	Remarks
PHPStan (level 7)	0 errors	All historical (pre-existing) errors resolved; relation and typing fixes applied
Automated tests (Pest)	20 passed / 81 assertions	Auth, RBAC, linked accounts, feed, teacher workflow, leaderboard suites
PHP code style (Pint)	Passed	Laravel preset, parallel checks
JS lint (ESLint)	Passed	classEase application only
JS format (Prettier)	Passed	classEase application only
JS type check (tsc)	Passed	classEase application only
Client production build	Passed	TypeScript + Vite build
Full-stack smoke test	Passed	Register / login / me / classes via nginx; SPA via Vite proxy
Docker stack	Up and serving	MySQL, Redis, app, queue, client, nginx

Test account details and the SQLite in-memory test invocation are described in Chapter 5.

APPENDIX C: REQUIRED SUBMISSION ATTACHMENTS

The following attachments are required for final submission. They are institution-generated documents and therefore cannot be produced by this report. Placeholders exist in the front matter of this document.

- Institution-approved project proposal (signed) — see front-matter placeholder.
- Plagiarism / similarity report issued by the institution's approved tool — see front-matter placeholder.
- Completed certificate, declaration and acknowledgement pages with the required signatures and college seal.
- Any additional institutional checklists (seminar evaluation, viva voce sheet, CD/journal submission form).